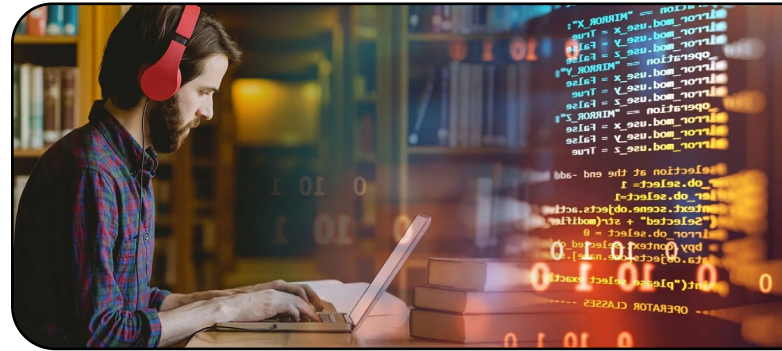


➤ Formularios – form

Creando el formulario para contacto



Pasos

Es el momento de crear nuestro primer formulario, el cual nos va a servir de formulario de contacto para nuestro blog.

Para el ejemplo, lo vamos a crear como una aplicación aparte, sin embargo, tu puedes hacerlo creando el modelo en una aplicación ya creada (en este momento la única aplicación creada es "posts", por lo que si quisieras podrías hacerlo ahí, sin embargo, para el ejemplo, te recomendamos seguir nuestros mismos pasos).

Vamos a posicionarnos en la carpeta apps y ejecutamos en consola: **django-admin startapp contacto**

```
(entorno) C:\Users\Pc\Proyecto Web\blog\apps>django-admin startapp contacto
```

Vamos a cargar el "models.py" de la aplicación creada.

```
apps > contacto > models.py > ...
1 from django.db import models
2 from django.utils import timezone
3
4 # Create your models here.
5
6 class Contacto(models.Model):
7     nombre_apellido = models.CharField(max_length=120)
8     email = models.EmailField()
9     asunto = models.CharField(max_length=50)
10    mensaje = models.TextField()
11    fecha = models.DateTimeField(default=timezone.now)
12
13    def __str__(self):
14        return self.nombre_apellido
15
```

Definimos campos en general usados como para cualquier formulario de contacto.

Lo siguiente es cambiar el acceso en "apps.py" para decirle que busque en la carpeta "apps".

```
models.py  apps.py  X
apps > contacto > apps.py > ...
1 from django.apps import AppConfig
2
3
4 class ContactoConfig(AppConfig):
5     default_auto_field = 'django.db.models.BigAutoField'
6     name = 'apps.contacto'
7
```

Cambiamos

Lo registramos en "admin.py"

```
apps > contacto > admin.py > ...
1 from django.contrib import admin
2 from .models import Contacto
3
4 # Register your models here.
5
6
7 @admin.register(Contacto)
8 class ContactoAdmin(admin.ModelAdmin):
9     list_display = ('id', 'nombre_apellido', 'email', 'asunto', 'fecha')
10
```

El "list_display", lo configuramos para que aparezcan todos los campos definidos en nuestro modelo.

Vamos a "settings.py" y declaramos nuestra nueva aplicación.

```
ls.py  apps.py  admin.py  settings.py  X
configuraciones > settings.py > ...
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

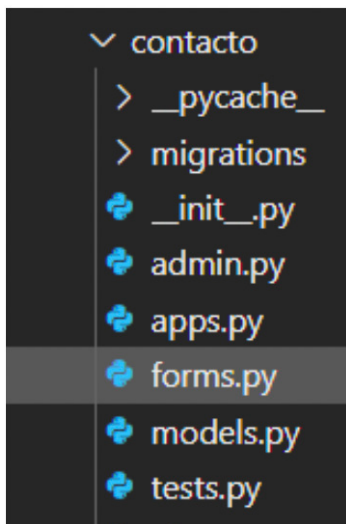
    'apps.posts',
    'apps.contacto',
]
```

¡No olvides de ir guardando todos los cambios!
Con esto hecho, es momento de realizar por consola las migraciones.

```
(entorno) C:\Users\Pc\Proyecto Web\blog>python manage.py makemigrations
Migrations for 'contacto':
  apps\contacto\migrations\0001_initial.py
    - Create model Contacto

(entorno) C:\Users\Pc\Proyecto Web\blog>python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contacto, contenttypes, posts, sessions
Running migrations:
  Applying contacto.0001_initial... OK
```

Es momento de crear nuestro formulario. Esto lo vamos a hacer dentro de la carpeta de nuestra aplicación como un nuevo archivo con el nombre de "forms.py".



Dentro de este archivo vamos a cargar todos los campos que declaramos en nuestro modelo pero con la herencia de un modelo de formulario propio de Django.

```
forms.py
blog > apps > contacto > forms.py > ...
1 from django import forms
2 from .models import Contacto
3
4 class ContactoForm(forms.ModelForm):
5     class Meta:
6         model = Contacto
7         fields = ['nombre_apellido', 'email', 'asunto', 'mensaje']
8
```

En las importaciones tenemos que traer "forms" desde Django y el modelo "Contacto".

Crearemos la clase, la cual nosotros pusimos como "ContactoForm", pero puede llevar cualquier nombre, sin embargo, te sugerimos que mantengas este tipo de referencias para que luego sea más fácil leer el código. Este formulario va a heredar de "ModelForm" a través de "form". Luego definimos la clase "Meta" especificando el modelo y los campos.

Ahora definiremos la view para este formulario:

```
views.py
apps > contacto > views.py > ...
1 from .forms import ContactoForm
2 from django.contrib import messages
3 from django.views.generic import CreateView
4 from django.urls import reverse_lazy
5
6
7 class ContactoUsuario(CreateView):
8     template_name = 'contacto/contacto.html'
9     form_class = ContactoForm
10    success_url = reverse_lazy('index')
11
12    def form_valid(self, form):
13        messages.success(self.request, 'Consulta enviada.')
14        return super().form_valid(form)
15
```

Las importaciones que vamos a tener son las del form "ContactoForm", luego vamos a importar "messages" para mostrar un mensaje luego de enviar el contacto, también traemos otra vista que nos provee Django que es "CreateView" y por último usaremos "reverse_lazy" para que una vez enviado el contacto, nos devuelva a la página "index".

En este caso usamos "reverse_lazy" ya que utilizamos una vista de clases, sin embargo, puedes usar otros atributos url de vistas genéricas de Django. Te recomendamos leer la documentación de Django para que puedas comprender mejor estos atributos url (<https://docs.djangoproject.com/en/4.2/ref/urllresolvers/>)

También debemos aclarar que esto es una vista de ejemplo muy básica. Si quieres hacer mejoras podrías usar el método "get_form_kwargs()" para pasar el request al formulario y así poder acceder al email del usuario autenticado si lo hay, también podrías sobrescribir el método "get_success_url()" para pasar el id del objeto creado al url de éxito y así poder mostrar los detalles del contacto o podrías usar el método "form_send_email()" para enviar el correo electrónico al destinatario y así separar la lógica de la vista del modelo.

Puede haber muchas mejoras pero te lo dejamos para que investigues (todo está en la documentación de Django) y poder hacer un mejor código con más funcionalidad.

Como puedes observar en la "view.py" la plantilla que vamos a usar la creamos con el nombre "contacto.html" dentro de una subcarpeta "contacto" que se encuentra en la carpeta "templates" - Estamos siguiendo la estructuración de carpetas que hicimos con los posts.

Ahora crearemos la url, pero tenemos que crear el archivo "urls.py" dentro de nuestra aplicación, ya que si recuerdas, este archivo no se crea con la creación de la aplicación. Lo creamos y definimos el acceso:

```
forms.py views.py urls.py _\contacto urls.py blog X contacto.html
blog > urls.py > ...
17 from django.contrib import admin
18 from django.urls import path, include
19 from .views import index
20 from django.conf import settings
21 from django.conf.urls.static import static
22 from django.contrib.staticfiles.urls import staticfiles_urlpatterns
23
24
25 urlpatterns = []
26 path('admin/', admin.site.urls),
27 path('', index, name='index'),
28 path('', include('apps.posts.urls')),
29 path('', include('apps.contacto.urls')),
30 ]
31 urlpatterns += static(settings.STATIC_URL, document_root=settings.STATIC_ROOT)
32 urlpatterns += staticfiles_urlpatterns()
33 urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

Es momento de completar el template contacto.

```
templates > contacto > contacto.html > ...
1 {% extends 'base.html' %}
2 {% load static %}
3
4 {% block contenido %}
5
6 <center>
7     <form method="POST" style="margin: 200px;">
8         <h1>Formulario de contacto</h1>
9         <br>
10        <table>
11            {% csrf_token %}
12            {{ form }}
13        </table>
14        <input type="submit" value="enviar">
15    </form>
16 </center>
17
18 {% endblock %}
```

Esta es una forma muy simple de completarlo donde te damos la sintaxis básica de lo fundamental que debe ir en el html que son:

- form method="POST" ----> para poder usar POST que envía los datos en el cuerpo de la solicitud y no en la URL. El método POST es el más común para enviar formularios, especialmente cuando se trata de datos sensibles o que modifican el estado del servidor.

Y antes de comenzar a completar el template "contacto.html" vamos a declarar esta url en el archivo "urls.py" principal.

Y antes de comenzar a completar el template "contacto.html" vamos a declarar esta url en el archivo "urls.py" principal.

- {% csrf_token %} es un tag de Django que protege el formulario contra ataques de falsificación de solicitudes entre sitios (CSRF, por sus siglas en inglés). Estos ataques ocurren cuando un sitio malicioso contiene un enlace, un botón o algún código javascript que intenta realizar alguna acción en tu sitio web, usando las credenciales de un usuario que visita el sitio malicioso en su navegador.
- {{ form }} trae los campos declarados en el formulario creado mediante la herencia del form que trajimos de Django.

Y como no estamos usando ni JS ni sweetalert o alguna otra forma de mostrar el mensaje de "Consulta enviada", en reverse_lazy le decimos que solo quede en "contacto" para que podamos visualizar el mensaje. Si recuerdas como lo hicimos en "posts", para acceder mediante esta declaración "apps.contacto:contacto", debemos declarar el nombre en "urls.py de nuestra aplicación "contacto":

Ahora si probamos, podemos enviar el formulario de contacto, queda recepcionado en el servidor y vuelve al "index", sin embargo, para usar el mensaje que definimos en la views mediante messages.success(self.request, 'Consulta enviada.') tenemos que agregar algo más al html y definir un contexto en la view:

```
6 <form method="POST" style="margin: 200px;">
7 <h1>Formulario de contacto</h1>
8 <br>
9 {% if messages %}
10     {% for message in messages %}
11         <div class="message {{ message.tags }}"> {{ message }} </div>
12     {% endfor %}
13 {% endif %}
14 <table>
15     {% csrf_token %}
16     {{ form }}
17 </table>
```

Lo que hacemos es introducir código Python para que si hay mensajes, haga un recorrido en "messages" y lo muestre. En views debemos agregar un contexto:

```
apps > contacto > views.py > ...
4 from django.urls import reverse_lazy
5
6
7 class ContactoUsuario(CreateView):
8     template_name = 'contacto/contacto.html'
9     form_class = ContactoForm
10     success_url = reverse_lazy('apps.contacto:contacto')
11
12     def get_context_data(self, **kwargs):
13         context = super().get_context_data(**kwargs)
14         context['request'] = self.request
15         return context
16
17     def form_valid(self, form):
18         messages.success(self.request, 'Consulta enviada.')
19         return super().form_valid(form)
20
```


> Formularios – form

Mostrando resultados



```
views.py | urls.py | contacto.html | forms.py
apps > contacto > urls.py > ...
1 from django.urls import path
2 from . import views
3
4 app_name = 'apps.contacto'
5
6 urlpatterns = [
7     path('contacto/', views.ContactoUsuario.as_view(), name='contacto'),
8 ]
```

Volvemos a repetir, no olvides guardar cada modificación hecha, y ahora antes de probar y que quede todo accesible, en “base.html” vamos a hacer la referencia para poder acceder desde el Navbar a “Contacto”:

```
views.py | urls.py | base.html | contacto.html | forms.py
templates > base.html > html > body > header > nav > div.nav-content > ul.nav-links >
31 </div>
32 <ul class="nav-links">
33 <li><a href="{% url 'index' %}">Inicio</a></li>
34 <li><a href="{% url 'apps.posts.posts' %}">Posts</a></li>
35 <li><a href="{% url 'apps.contacto:contacto' %}">Contacto</a></li>
36 <li><a href="{% url 'apps.contacto:contacto' %}">Contacto</a></li>
37 </ul>
38 </div>
39 </nav>
40
41 {% block contenido %}
42
43 {% endblock %}
```

Ahora desde nuestro “index” podemos acceder a Contacto. Enviamos una consulta y nos tiene que aparecer el mensaje de “Consulta enviada”.

Y si accedemos al “admin” de Django, podremos ver el mensaje enviado.

Con esto realizamos todos los pasos para crear nuestro primer formulario. Algo parecido vamos a hacer para crear otros formularios como el de agregar posts para un usuario con acceso a esto, o para comentarios, también usaremos forms.

Hasta aquí llegamos con esta parte del apunte, en el próximo te mostraremos más de nuestra página formateada con css y js, por lo que si te animas, ¡también puedes hacerlo!

> Usuarios

Creando app usuario



Pasos

Para seguir integrando este blog, en este apartado veremos el uso de usuarios que nos provee Django. Para el contenido de este proyecto podemos usar User o AbstractUser o AbstractBaseUser.

Para nuestro caso vamos a usar AbstractUser, por lo que vamos a comenzar creando al app usuario para luego seguir con el modelo.

```
(entorno) C:\Users\Pc\Proyecto Web\blog\apps>django-admin startapp usuario
```

Ahora dentro de la aplicación usuario vamos a ir al "models.py"

```
apps > usuario > models.py >
1 from django.db import models
2 from django.urls import reverse
3 from django.contrib.auth.models import AbstractUser
4
5 # models
6
7 class Usuario(AbstractUser):
8     imagen = models.ImageField(null=True, blank=True, upload_to='usuario', default='usuario/user-default.png')
9
10     def get_absolute_url(self):
11         return reverse('index')
```

Como puedes ver hacemos las importaciones pertinentes que usaremos. Dentro de estas está AbstractUser.

Al heredar de AbstractUser, el modelo Usuario incluye todos los campos y métodos del modelo de usuario predeterminado de Django, como nombre de usuario, correo electrónico y contraseña.

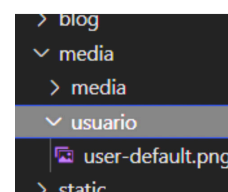
Además, en el modelo Usuario agregamos un campo adicional llamado "imagen". Este campo permite a los usuarios subir una imagen de perfil.

El campo imagen tiene las siguientes propiedades:

- null=True: Esto significa que el campo puede ser nulo en la base de datos.
- blank=True: Esto significa que el campo puede estar en blanco en los formularios.
- upload_to='usuario': Esto especifica el subdirectorio dentro del directorio MEDIA_ROOT donde se guardarán las imágenes subidas.
- default='usuario/user-default.png': Esto especifica la imagen predeterminada que se utilizará si el usuario no sube una imagen.

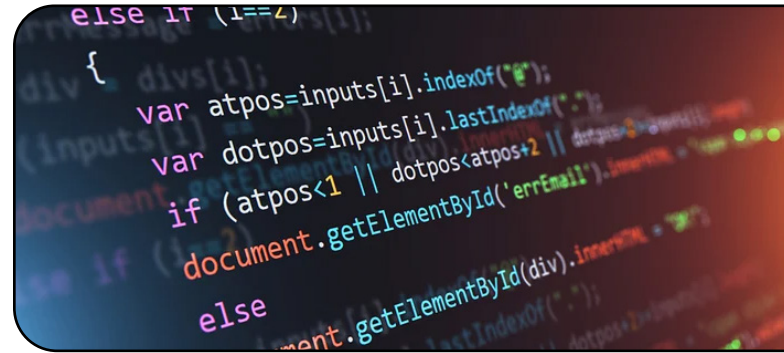
Finalmente, el modelo Usuario define un método llamado "get_absolute_url()". Este método devuelve la URL absoluta del objeto de usuario. En este caso, el método devuelve la URL correspondiente a la vista con el nombre 'index'. Esto significa que, cuando se llama al método get_absolute_url() en un objeto de usuario, se redirigirá al usuario a la página principal de la aplicación.

Para las imágenes de los usuarios, creamos una nueva carpeta "usuario" dentro de la carpeta "media" donde agregamos una imagen que va a ser la que lleve por defecto un usuario que se registre, si es que no sube una imagen propia, la cual, en tal caso, se guardará dentro de la carpeta "media/usuario".



> Usuarios

Configuraciones de app usuario



Pasos

Registraremos este modelo en el admin de Django, por lo que abrimos "admin.py" de nuestra app usuario:

```
admin.py x
apps > usuario > admin.py
1 from django.contrib import admin
2 from .models import Usuario
3
4 # Register your models here.
5
6 admin.site.register(Usuario)
7
```

Luego en apps.py cambiaremos la ruta de acceso:

```
apps.py
apps > usuario > apps.py > ...
1 from django.apps import AppConfig
2
3 class UsuarioConfig(AppConfig):
4     default_auto_field = 'django.db.models.BigAutoField'
5     name = 'apps.usuario'
6
```

Lo próximo es configurar el settings.py agregando una nueva línea:

```
28 ALLOWED_HOSTS = []
29
30 AUTH_USER_MODEL = 'usuario.Usuario'
31
32 # Application definition
```

Luego agregamos "apps.usuario" en nuestras aplicaciones:

```
34 INSTALLED_APPS = [
35     'django.contrib.admin',
36     'django.contrib.auth',
37     'django.contrib.contenttypes',
38     'django.contrib.sessions',
39     'django.contrib.messages',
40     'django.contrib.staticfiles',
41
42     'apps.posts',
43     'apps.contacto',
44     'apps.usuario',
45 ]
```

Ahora si quisieramos hacer las migraciones, nos arrojaría un error ya que hasta este momento estabamos utilizando el usuario "User" de Django, sin embargo, al crear un usuario "AbstractUser", estaríamos reemplazando el usuario actual de Django por lo que se generaría una inconsistencia en la base de datos. Por lo que es momento de enlazar con unos de los temas que los dejamos hasta este momento para que, con el nuevo tipo de usuario, puedas comenzar a usar la base de datos de MySQL. Pero antes de crear esta base de datos, y por si quieres seguir usando sqlite3, vamos a borrar los datos que ya tiene para poder usar AbstractUser.

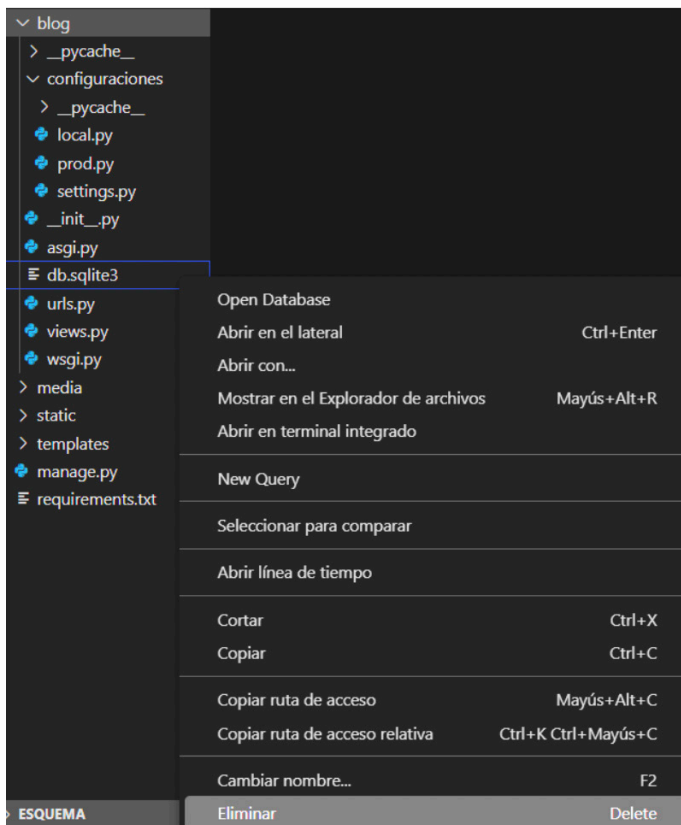
> Base de datos

Borrado de sqlite3



Pasos

Vamos a dirigirnos a la carpeta "blog/blog" y eliminamos nuestra base de datos manualmente con click derecho - Eliminar:



```
(entorno) C:\Users\Pc\Proyecto Web\blog>py manage.py makemigrations
No changes detected

(entorno) C:\Users\Pc\Proyecto Web\blog>py manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contacto, contenttypes, posts, sessions, usuario
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0001_initial... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying usuario.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contacto.0001_initial... OK
  Applying posts.0001_initial... OK
  Applying posts.0002_rename_categorias_categoria_rename_posts_post... OK
  Applying sessions.0001_initial... OK
```

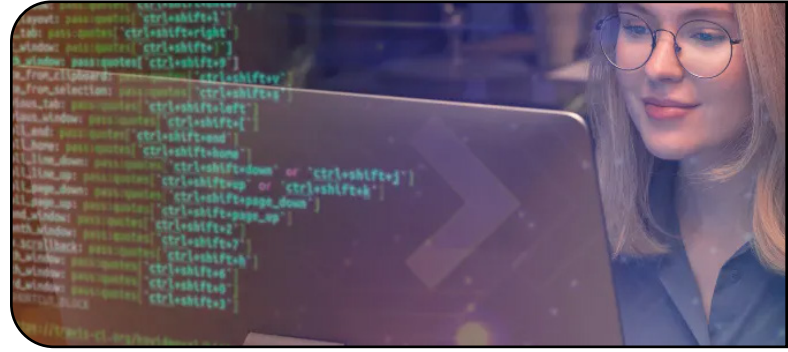
Ahora ya tenemos limpia nuestra base de datos y con "AbstractUser" funcionando en lugar de "User". Para poder volver a ingresar al admin de Django (<http://127.0.0.1:8000/admin/>) debemos crear nuevamente el superusuario como lo hicimos en la página 32. Hecho eso, ya podemos volver a acceder a nuestra base de datos sqlite3, sin embargo, como te mencionamos anteriormente, vamos a comenzar a usar MySQL y para esto te vamos a mostrar 2 formas de crear la base de datos.

Nos saldrá una ventana donde nos dirá si estamos seguros que queremos eliminar y presionamos en "Mover a la papelera de reciclaje".

Una vez eliminada la base de datos de sqlite3, podemos realizar "makemigrations" y "migrate":

> Base de datos

Mysql – creación y enlace con el proyecto



Podemos hacerlo desde la consola o desde Workbench. Vamos a hacerlo desde la consola primero para que puedas ver las dos formas.

Primero vamos a abrir una consola cmd, ya sea desde Windows (Inicio – cmd) o desde la terminal en la que estamos trabajando desde VSC y ejecutamos `mysql -u root -p`

Si te llega a salir un error como este:

```
"mysql" no se reconoce como un comando interno o externo,
programa o archivo por lotes ejecutable.
```

Debes realizar la siguiente configuración del sistema ya que esto se debe a que el "Path" del sistema no tiene agregado MySQL.

Abre el Panel de control y busca la opción "Sistema".

Haz clic en "Configuración avanzada del sistema" en el menú de la izquierda.

Haz clic en el botón "Variables de entorno".

En la sección "Variables del sistema", busca la variable "Path" y haz clic en "Editar".

Haz clic en "Nuevo" y agrega la ruta del directorio que contiene las herramientas de línea de comandos de MySQL (por ejemplo, `C:\Program Files\MySQL\MySQL Server X.Y\bin`).

Haz clic en "Aceptar" para guardar los cambios y cerrar todas las ventanas.

Si no hay ningún problema, la consola te pedirá la contraseña configurada.

```
Símbolo del sistema - mysql -u root -p
Microsoft Windows [Versión 10.0.19045.3208]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\PC>mysql -u root -p
Enter password: ****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 11
Server version: 8.0.33 MySQL Community Server - GPL

Copyright (c) 2000, 2023, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

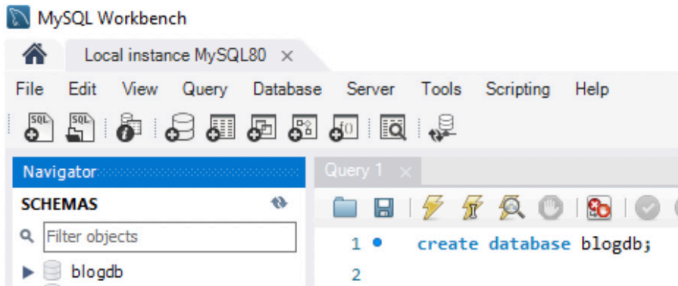
mysql>
```

Ahora ya podemos crear la base de datos como ya lo sabés hacer, con el comando `CREATE` seguido del nombre de la base de datos que vamos a crear. En este caso, vamos a llamarla con el mismo nombre del proyecto y agregando las letras db al final y luego para verla vamos a ejecutar el comando `"show databases;"` con el cual se mostrará la base de datos creada. Al final salimos con `"exit"`.

```
Type 'help;' or '\h' for help. Type
mysql> create database blogdb;
Query OK, 1 row affected (0.00 sec)

mysql> show databases;
+-----+
| Database |
+-----+
| blogdb   |
+-----+
```

Pero si queremos usar Workbench vamos a abrirlo y ejecutamos el mismo comando para crear nuestra base de datos.

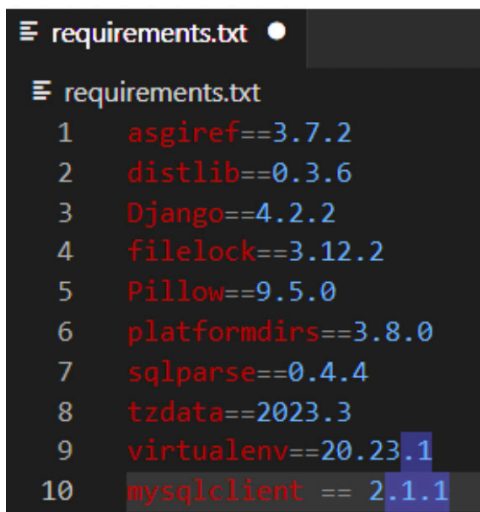


Ahora es momento de modificar el settings.py de nuestro proyecto para comenzar a usar esta base de datos, pero, en nuestro caso, ahora comenzaremos a usar "local.py" de las configuraciones que habíamos hecho al principio. En este archivo declararemos que vamos a usar MySQL. Para ello necesitamos instalar la dependencia de MySQL para Django.

Vamos a la terminal y ejecutamos:
pip install mysqlclient

C:\Users\Pc\Proyecto Web\blog>pip install mysqlclient
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: mysqlclient in c:\users\pc\appdata\local\programs\python\python311\site-packages (2.1.1)

y lo agregamos a requirements.txt. En este caso con la versión que se instaló, pero si usas otra versión, asegurate que sea la misma versión que se instaló al momento de hacer el proyecto.

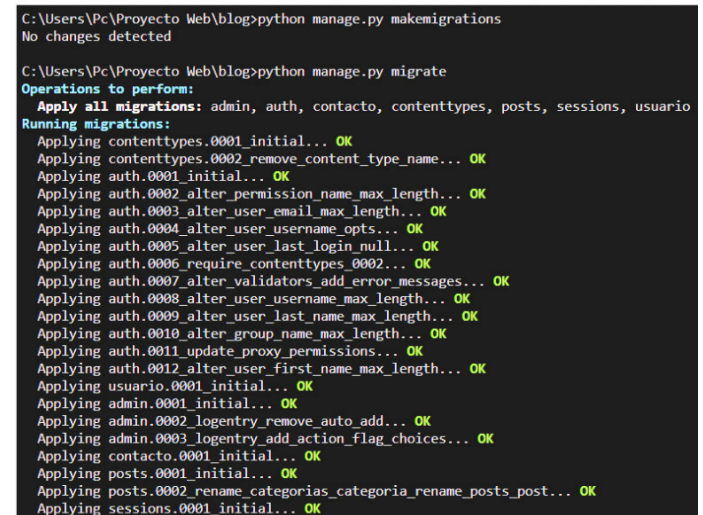


Nos dirigimos a "local.py" y agregamos las siguientes líneas de código poniendo el password que usaremos (en nuestro caso usamos root).

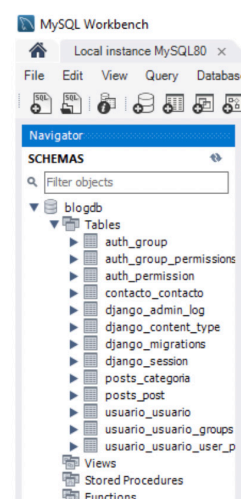
Luego debes dirigirte a "settings.py" y borrar la base de datos declarada ahí:



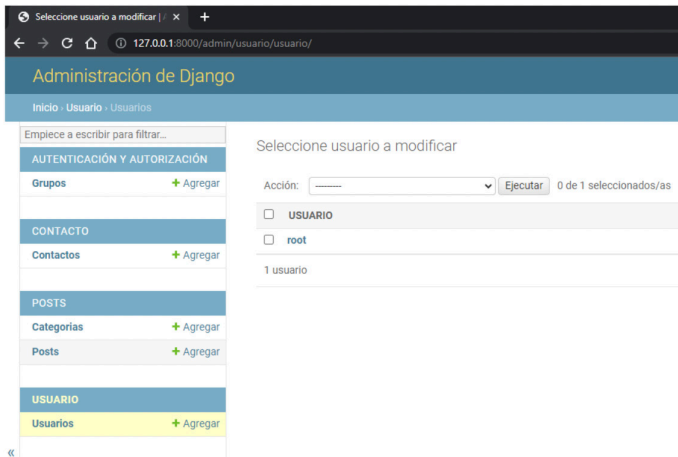
Guardamos todo y realizamos las migraciones:



Si miramos desde Workbench ahora (o desde la consola cmd), vamos a ver que se crearon correctamente las tablas:



Ahora nuevamente debemos crear el superuser para esta base de datos (página 32) para poder acceder al admin de Django.



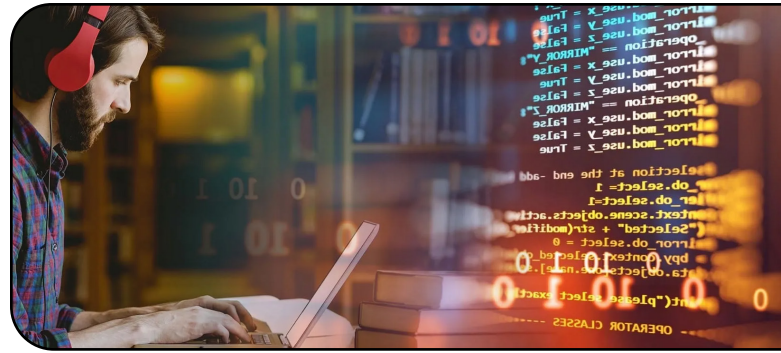
No olvides que teníamos el servidor detenido para hacer estas modificaciones y tienes que volver a correrlo para ingresar al admin nuevamente.

Como verás ahora se separó Usuarios de Grupos y esto es porque ahora estamos usando el AbstractUser para declarar un nuevo usuario en las apps, en vez de User que estuvimos usando hasta el momento.

Ahora podemos seguir personalizando nuestro usuario.

> Usuario

Forms – view – url – template para usuarios



```
EXPLORADOR  ...  forms.py
▼ BLOG
  ▼ apps
    > contacto
    > posts
    ▼ usuario
      > __pycache__
      > migrations
      > __init__.py
      > admin.py
      > apps.py
      > forms.py
      > models.py
      > tests.py
      > views.py
    > blog
    > media
    > static
    > templates
    > manage.py
    > requirements.txt

apps > usuario > forms.py > ...
1  from .models import Usuario
2  from django.contrib.auth.forms import UserCreationForm
3  from django import forms
4  from django.contrib.auth import authenticate, login
5
6
7  class RegistroUsuarioForm(UserCreationForm):
8
9      class Meta:
10         model = Usuario
11         fields = ['username', 'first_name', 'last_name', 'password1', 'password2', 'email', 'imagen']
12
13
14
15  class LoginForm(forms.Form):
16     username = forms.CharField(label='Nombre de usuario')
17     password = forms.CharField(label='Contraseña', widget=forms.PasswordInput)
18
19     def login(self, request):
20         username = self.cleaned_data.get('username')
21         password = self.cleaned_data.get('password')
22         user = authenticate(request, username=username, password=password)
23         if user:
24             login(request, user)
25
```

Como puedes ver este código es un ejemplo de cómo se pueden crear formularios en Django para el registro y el inicio de sesión de usuarios.

Primero, se importan las clases necesarias de Django y del modelo Usuario. Luego, se define la clase "RegistroUsuarioForm" que hereda de "UserCreationForm" y se especifica que el modelo a utilizar es "Usuario" y los campos que se van a incluir en el formulario. Después, se define la clase "LoginForm" que hereda de forms.Form y se definen los campos para el nombre de usuario y la contraseña.

Finalmente, se define el método "login" que autentica al usuario e inicia sesión si las credenciales son correctas.

El siguiente paso, como lo venimos haciendo, es crear las vistas. En estas vistas basadas en clases que vamos a usar, vamos a estar heredando desde lo que nos provee Django (como habrás leído sobre estas vistas cuando te lo recomendamos en la página 51).


```
apps > usuario > views.py > ...
1  from .forms import RegistroUsuarioForm
2  from django.contrib.auth.views import LoginView, LogoutView
3  from django.views.generic import CreateView
4  from django.contrib import messages
5  from django.shortcuts import redirect
6  from django.urls import reverse
7
8  # Create your views here.
9
10 class RegistrarUsuario(CreateView):
11     template_name = 'registration/registrar.html'
12     form_class = RegistroUsuarioForm
13
14     def form_valid(self, form):
15         messages.success(self.request, 'Registro exitoso. Por favor, inicia sesión.')
16         form.save()
17
18         return redirect('apps.usuario:registrar')
19
20 class LoginUsuario(LoginView):
21     template_name = 'registration/login.html'
22
23     def get_success_url(self):
24         messages.success(self.request, 'Login exitoso')
25
26         return reverse('apps.usuario:login')
27
28
29 class LogoutUsuario(LogoutView):
30     template_name = 'registration/logout.html'
31
32     def get_success_url(self):
33         messages.success(self.request, 'Logout exitoso')
34
35         return reverse('apps.usuario:logout')
36
```

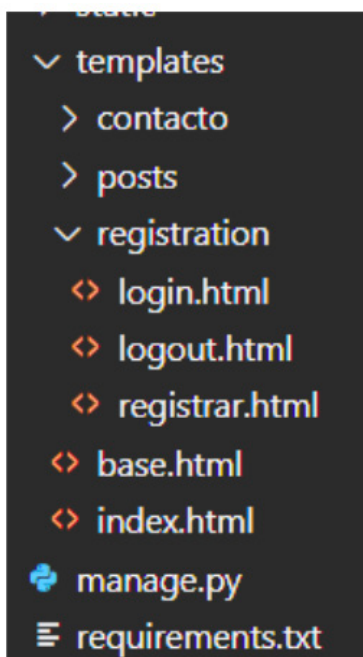
Para registrar el usuario heredamos de la vista `CreateView`, como lo hicimos con “contacto” y, de la misma forma también usamos el método para guardar si el formulario es válido con un mensaje incluido. La redirección de la página la dejamos en el registro para poder ver el mensaje de éxito, sin embargo, cuando mejores el código en tu proyecto puedes usar, por ejemplo, `SweetAlert` para mostrar un mensaje emergente del registro exitoso y que se redirija

a la página de iniciar sesión (puedes buscar en la documentación de Django e investigar sobre mensajes: <https://docs.djangoproject.com/en/4.2/ref/contrib/messages/> y sobre mensajes `SweetAlert`: <https://lipis.github.io/bootstrap-sweetalert/>). Lo mismo puedes hacer para “login”, “logout” o “contacto” e incluso para “comentarios” (app que crearemos luego).

Luego para "login" heredamos de la vista que nos proporciona Django "LoginView" donde solo agregamos un mensaje de "Login exitoso" y lo mismo hacemos para "logout". En ambos casos volvemos a quedar en el mismo template para ver los mensajes, pero la idea es que se redirija a la página de inicio, o si estamos en una cierta página, como sería la de un post en particular y queremos hacer un comentario pero debemos loguearnos o registrarnos, debería volver a la misma página del post en el que estamos. Puedes investigar sobre "request.path" (?next={{ request.path }} - <https://developer.mozilla.org/en-US/docs/Learn/Server-side/Django/Authentication>).

Esto último se agrega sobre los templates, por lo que en estos enlaces que te dejamos, puedes investigar, sin embargo, ahora nos toca realizar los templates (de forma muy básica) para poder visualizar lo que estuvimos haciendo.

Como pudiste ver en las views.py vamos a usar los templates registrar.html, login.html y logout.html que se encuentran en la carpeta "registration" la cual es una convención de Django ya que estamos usando las vistas que nos provee:



Dentro de estas tres plantillas vamos a usar formularios como lo habíamos hecho con "contacto", ya que, para todos necesitamos trabajar con la base de datos, por lo que llevarán las mismas bases.

Además, como lo hicimos en "contacto" agregaremos código Python para recorrer los mensajes y nos devuelva los mismos. Te dejamos las vistas http para que veas la forma básica y luego los templates de cada uno.



Registro de usuarios

Registro exitoso. Por favor, inicia sesión.

Nombre de usuario:

Nombre:

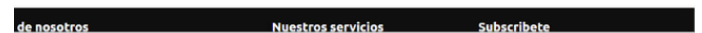
Apellido:

Contraseña:

Confirmación de contraseña:

Dirección de email:

Imagen:



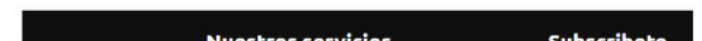
Iniciar sesión

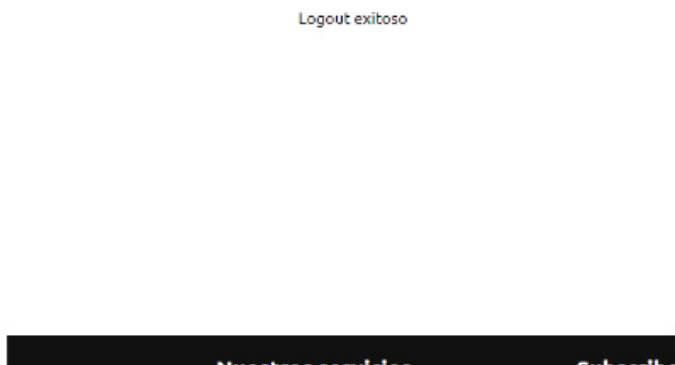
Login exitoso

Nombre de usuario:

Contraseña:

Todavía no está registrado? Regístrate





Por el momento venimos escribiendo el nombre de la url de forma manual en el http (<http://127.0.0.1:8000/logout/>), pero luego haremos los enlaces y te lo mostraremos. Pero antes te dejamos las plantillas html:

```
templates > registration > registrar.html > ...
1  {% extends 'base.html' %}
2  {% load static %}
3
4  <title>{% block title %} Registro de usuario {% endblock title %}</title>
5
6  {% block contenido %}
7  <center>
8
9      <form method="POST" enctype="multipart/form-data" style="margin-top: 300px; margin-bottom: 100px;">
10         <h1 class="h1">Registro de usuarios</h1>
11         <br>
12         <div class="messages">
13             {% for message in messages %}
14                 <table{% if message.tags %} class="{{ message.tags }}" {% endif %}>{{ message }}</table>
15             {% endfor %}
16         </div>
17         <br>
18         {% csrf_token %}
19         <div class="container-fluid">
20             <table>{{ form }}</table>
21         </div>
22         <br><br>
23         <button type="submit" class="btn btn-success btn-lg px-3">Registrar</button>
24     </form>
25
26 </center>
27 {% endblock %}
```

Para registrar es muy importante que agregues `enctype="multipart/form-data"` ya que sin este no subirá la imagen cuando la agreguemos en el campo "imagen". El resto del código, como verás es muy similar al de "contacto", por lo que cada vez que queramos un formulario, usaremos básicamente esta estructura. Hay veces que necesitamos campos específicos y en esos casos utilizaremos "label_tag" (<https://docs.djangoproject.com/en/4.2/ref/forms/api/>).

```
templates > registration > logout.html > ...
1  {% extends 'base.html' %}
2  {% load static %}
3
4  {% block contenido %}
5  <center>
6  <div class="container" style="margin-top: 370px;">
7      <div class="messages">
8          {% for message in messages %}
9              <table{% if message.tags %} class="{{ message.tags }}" {% endif %}>{{ message }}</table>
10          {% endfor %}
11      </div>
12  </div>
13  </center>
14  {% endblock %}
```

Logout no llevará demasiado código ya que lo más importante aquí es mostrar el mensaje de logout. Django se ha encargado de realizar el logout correctamente.

```
templates > registration > login.html > ...
1  {% extends 'base.html' %}
2  {% load static %}
3
4  <title>{% block title %} Registro de usuario {% endblock title %}</title>
5
6  {% block contenido %}
7  <center>
8
9      <form method="POST" style="margin-top: 300px;">
10         <h1 class="h1">Iniciar sesión</h1>
11         <br>
12         <div class="messages">
13             {% for message in messages %}
14                 <table{% if message.tags %} class="{{ message.tags }}" {% endif %}>{{ message }}</table>
15             {% endfor %}
16         </div>
17         <br>
18         {% csrf_token %}
19         <div class="container">
20             <table>{{ form }}</table>
21         </div>
22         <a style="color: black;" href="{% url 'apps.usuario:registrar' %}">Todavía no está registrado? Regístrate</a>
23         <br>
24         <button type="submit" class="btn btn-success btn-lg px-3">Iniciar sesión</button>
25     </form>
26
27 </center>
28
29 {% endblock %}
```

Y para login lo que hicimos fue agregar un botón para que si el usuario no está registrado, pueda acceder directamente al registro de usuario.

Ahora solo nos resta configurar en base.html los accesos desde el Navbar:

```
templates > base.html > html > body > header > nav > div.nav-content > ul.nav-links > li
31 </div>
32 <ul class="nav-links">
33     <li><a href="{% url 'index' %}">Inicio</a></li>
34     <li><a href="#">Acerca de nosotros</a></li>
35     <li><a href="{% url 'apps.posts:posts' %}">Posts</a></li>
36     <li><a href="{% url 'apps.contacto:contacto' %}">Contacto</a></li>
37     <li><a href="{% url 'apps.usuario:login' %}">Login</a></li>
38 </ul>
39 </div>
```

Con esto podremos acceder a loguearnos con nuestro usuario, pero si el usuario no se encuentra registrado, puede acceder desde el mismo template de login al registro de usuarios.

Sin embargo, nos falta una cláusula más para que se habilite un botón de "logout" cuando el usuario está logueado, y eso lo vamos a hacer introduciendo un condicional a nuestro html:

```

31 </div>
32 <ul class="nav-links">
33 <li><a href="{% url 'index' %}">Inicio</a></li>
34 <li><a href="#">Acerca de nosotros</a></li>
35 <li><a href="{% url 'apps.posts.posts' %}">Posts</a></li>
36 <li><a href="{% url 'apps.contacto:contacto' %}">Contacto</a></li>
37 {% if user.is_authenticated %}
38 <li><a href="{% url 'apps.usuario:logout' %}">Logout</a></li>
39 {% else %}
40 <li><a href="{% url 'apps.usuario:login' %}">Login</a></li>
41 {% endif %}
42 </ul>
43 </div>
44 </nav>

```

Mediante algunas pequeñas configuraciones, vamos a tener el reseteo completo del password del usuario.

Lo primero va a ser agregar en la urls.py principal lo siguiente:

```

18 from django.urls import path, include
19 from .views import index
20 from django.conf import settings
21 from django.conf.urls.static import static
22 from django.contrib.staticfiles.urls import staticfiles_urlpatterns
23
24
25 urlpatterns = [
26     path('admin/', admin.site.urls),
27     path('', index, name='index'),
28     path('', include('apps.posts.urls')),
29     path('', include('apps.contacto.urls')),
30     path('', include('apps.usuario.urls')),
31     path('', include('django.contrib.auth.urls'))]
32 ]+static(settings.STATIC_URL, document_root=settings.STATIC_ROOT)
33 urlpatterns += staticfiles_urlpatterns()
34 urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
35

```

```

31 path('', include('django.contrib.auth.urls'))]

```

Lo que estamos haciendo en este momento, es decirle a Django que verifique que si el usuario está logueado o autenticado, el botón que tiene que aparecer es el de "Logout", pero si no está logueado, tiene que aparecer "Login".

Lo siguiente para finalizar con los usuarios es hacer los accesos de reseteo de password por si el usuario se ha olvidado la contraseña.

Esto lo vamos a hacer gracias a que Django por defecto ya tiene instalada la aplicación:

```

33
34 INSTALLED_APPS = [
35     'django.contrib.admin',
36     'django.contrib.auth',
37     'django.contrib.contenttypes',
38     'django.contrib.sessions',
39     'django.contrib.messages',
40     'django.contrib.staticfiles',
41
42     'apps.posts',
43     'apps.contacto',

```

Lo siguiente será crear las urls desde la aplicación usuario:

```

apps > usuario > urls.py > ...
1 from django.urls import path
2 from . import views
3 from .views import LoginUsuario
4 from django.contrib.auth import views as auth_views
5
6 app_name = 'apps.usuario'
7
8 urlpatterns = [
9     path('registrar/', views.RegistrarUsuario.as_view(), name='registrar'),
10    path('login/', LoginUsuario.as_view(), name='login'),
11    path('logout/', views.LogoutUsuario.as_view(), name='logout'),
12    path('password_reset/', auth_views.PasswordResetView.as_view(), name='password_reset'),
13    path('password_reset/done/', auth_views.PasswordResetDoneView.as_view(), name='password_reset_done'),
14    path('reset/<uidb64>/<token>', auth_views.PasswordResetConfirmView.as_view(), name='password_reset_confirm'),
15    path('reset/done/', auth_views.PasswordResetCompleteView.as_view(), name='password_reset_complete'),
16 ]

```


donde tendremos que agregar los "path" para "password_reset", "password_reset/done", "reset/<uidb64>/<token>/" y "reset/done/". Posterior a esto debemos crear las plantillas para mostrar los formularios y mensajes correspondientes:

- **registration/password_reset_form.html:** Esta plantilla se utiliza para mostrar el formulario donde los usuarios pueden ingresar su dirección de correo electrónico para solicitar un restablecimiento de contraseña.
- **registration/password_reset_done.html:** Esta plantilla se utiliza para mostrar un mensaje indicando que se ha enviado un correo electrónico con instrucciones para restablecer la contraseña.
- **registration/password_reset_email.html:** Esta plantilla se utiliza para generar el correo electrónico que se envía al usuario con instrucciones para restablecer su contraseña. Debe incluir un enlace a la URL `password_reset_confirm` con los argumentos `uidb64` y `token` proporcionados por la vista.
- **registration/password_reset_confirm.html:** Esta plantilla se utiliza para mostrar el formulario donde los usuarios pueden ingresar una nueva contraseña.
- **registration/password_reset_complete.html:** Esta plantilla se utiliza para mostrar un mensaje indicando que la contraseña se ha restablecido correctamente.

Por último, antes de crear y completar cada template, debemos configurar un correo electrónico

para que se realice el envío del formulario de reseteo de la contraseña. Esta configuración requiere un correo que usemos para hacer el envío del formulario. Si vamos a utilizar un correo distinto cuando trabajemos en producción, te recomendamos realizar la configuración en `local.py` de nuestras configuraciones y en `prod.py` el correo que utilizaremos cuando este proyecto se encuentre en producción.

Si vas a usar el mismo correo tanto para el entorno `local`, como para el entorno de producción, puedes poner esta configuración directamente en `settings.py`. Nosotros vamos a agregarla en `settings.py` para el ejemplo. Puedes agregar estas líneas de código en cualquier parte de tu archivo de configuración:

```
blog > configuraciones > settings.py > ...
30 AUTH_USER_MODEL = 'usuario.Usuario'
31
32 EMAIL_BACKEND = 'django.core.mail.backends.smtp.EmailBackend'
33 EMAIL_HOST = 'smtp.gmail.com'
34 EMAIL_PORT = 587
35 EMAIL_USE_TLS = True
36 EMAIL_HOST_USER = 'email_a_utilizar@gmail.com'
37 EMAIL_HOST_PASSWORD = 'contraseña de tu email'
38
39 # Application definition
40
41 INSTALLED_APPS = [
```

Asegurate de reemplazar por tu email y contraseña en esta parte.

Es momento de crear y completar nuestras plantillas para mostrar los formularios y mensajes. Lo vemos en la siguiente página.

```
<> password_reset_form.html ●
templates > registration > <> password_reset_form.html > ...
1  {% extends 'base.html' %}
2
3  {% block contenido %}
4      <h2>Restablecer contraseña</h2>
5      <form method="POST">
6          {% csrf_token %}
7          {{ form }}
8          <button type="submit">Enviar correo electrónico</button>
9      </form>
10 {% endblock %}
11 <center>
12     <div class="container" style="margin-top: 370px;">
13         <div class="messages">
14             {% for message in messages %}
15                 <table{% if message.tags %} class="{% message.tags %}"{% endif %}>{{ message }}</table>
16             {% endfor %}
17         </div>
18     </div>
19 </center>
```

```
<> password_reset_done.html ✕
templates > registration > <> password_reset_done.html > ...
1  {% extends 'base.html' %}
2
3  {% block contenido %}
4      <center>
5          <h2>Correo electrónico enviado</h2>
6          <p>Se ha enviado un correo electrónico con instrucciones para restablecer tu contraseña.</p>
7      </center>
8  {% endblock %}
```

```
<> password_reset_email.html ✕
templates > registration > <> password_reset_email.html
1  Hola, has solicitado restablecer tu contraseña. Haz clic en el siguiente enlace para completar el proceso:
2
3  {{ protocol }}://{{ domain }}{% url 'password_reset_confirm' uidb64=uid token=token %}
4
5  Si no has solicitado restablecer tu contraseña, ignora este correo electrónico.
6
7  Saludos,
8  El equipo de {{ site_name }}
```

Aquí no hay que cambiar nada. Django se encargará de todo.

- `{{ protocol }}`: Esta variable se reemplaza por el protocolo utilizado para generar el enlace de restablecimiento de contraseña (por ejemplo, "http" o "https").
- `{{ domain }}`: Esta variable se reemplaza por el nombre de dominio de tu sitio (por ejemplo, "www.example.com").
- `{% url 'password_reset_confirm' uidb64=uid token=token %}`: Esta etiqueta de plantilla genera la URL para la vista de confirmación de restablecimiento de contraseña. Las variables uidb64 y token se reemplazan automáticamente por los valores correctos para el usuario que solicitó el restablecimiento de contraseña.

En cuanto a la variable `{{ site_name }}`, esta variable se reemplaza por el nombre de tu sitio. Puedes definir el valor de esta variable en la configuración `SITE_NAME` de tu proyecto de Django. Por ejemplo, si deseas que el nombre de tu sitio sea "Mi sitio", puedes agregar la siguiente línea a tu archivo `settings.py`:

```

settings.py X
blog > configuraciones > settings.py > ...
38
39 SITE_NAME = 'Informatario'
40

```

```

password_reset_confirm.html X
templates > registration > password_reset_confirm.html > ...
1 {% extends 'base.html' %}
2
3 {% block contenido %}
4 <center>
5   <h2>Restablecer contraseña</h2>
6   <form method="POST">
7     {% csrf_token %}
8     {{ form }}
9     <button type="submit">Restablecer contraseña</button>
10  </form>
11 </center>
12 {% endblock %}

```

```

password_reset_complete.html
templates > registration > password_reset_complete.html > ...
1 {% extends 'base.html' %}
2
3 {% block contenido %}
4 <center>
5   <h2>Contraseña restablecida</h2>
6   <p>Tu contraseña se ha restablecido correctamente.</p>
7 </center>
8 {% endblock %}

```

Hecho esto, solo queda enlazar las urls correspondiente en el template de login, arriba o abajo de registrarse (lo que comunemente se hace) y con eso ya se completaría la parte básica de usuario.

¡Te lo dejamos para que lo hagas!